MDAnalysis Dashboard - Mini Workshop

August 20, 2026

MD ANALYSIS mdadash

Google Summer of Code

Presenter

Pardhav Maradani

- Undergraduate student from India
- Pursuing Bachelors in Computer Science and Engineering
  - Vellore Institute of Technology, Vellore (Class of ‘28)
- Pursuing BS in Data Science and Applications
  - Indian Institute of Technology, Madras
- Google Summer of Code Contributor to MDAnalysis org
  - [2025] Better interfacing of Blender and MDAnalysis
  - [2026] Dashboard for tracking MD simulation progress with the new streaming interface

Agenda

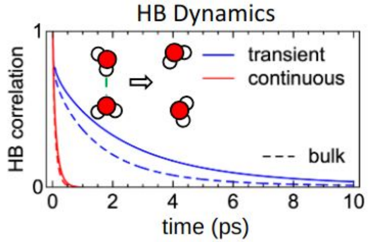
1. Streaming - Overview (5 min)
2. **mdadash** - Introduction (5 min)
3. Features (15 min)
4. Demo (20 min)
5. Interactive Activities (40 min)
6. Future Work (5 min)
7. Q & A (15 min)

Streaming - Overview

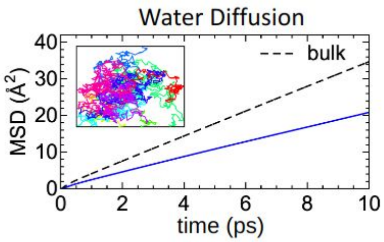
Why Streaming of MD trajectories?

- **“On-the-fly” Analysis of Trajectories**
  - **Analyze fast** (sub-picosecond) **processes** w/o writing large trajectory output files
    - Solvent Dynamics (HBs, diffusion)
    - Molecular Vibrations
    - Friction (force time correlations)
  - **Event Detection**
    - Monitor arbitrary collective variables
    - Weighted Ensemble methods
    - Water / ion flux through membrane
- **Control Output**
  - Save disk space & post processing
    - Store only data you need (Eg: unwrapped protein coordinates)

$$C_{\mathrm{HB}}(\tau) = \left\langle \frac{\hat{h}(t+\tau)\cdot\hat{h}(t)}{\hat{h}(t)\cdot\hat{h}(t)} \right\rangle_t$$

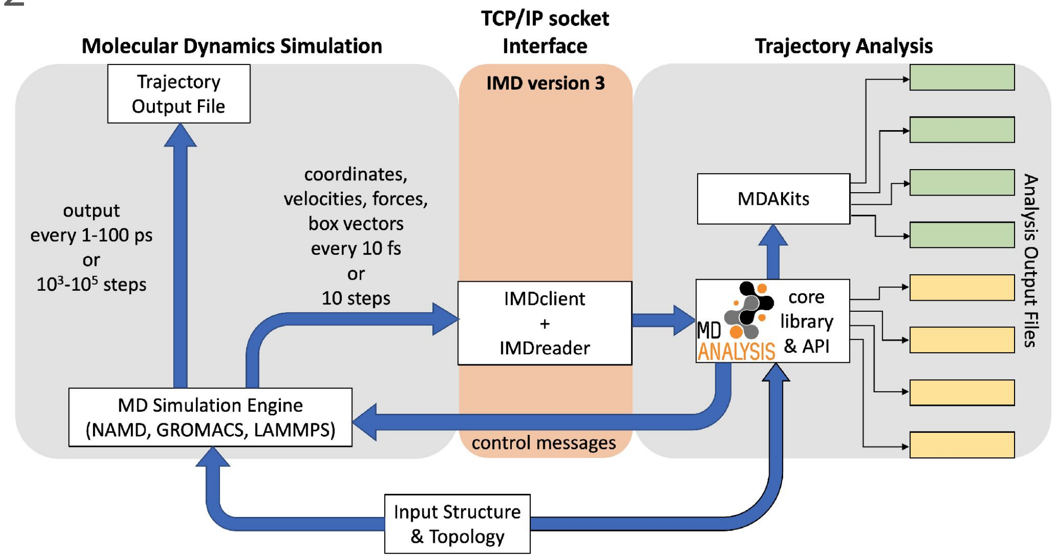


$$MSD(\tau) = \langle |\vec{x}(t+\tau) - \vec{x}(t)|^2 \rangle_t$$



IMDv3

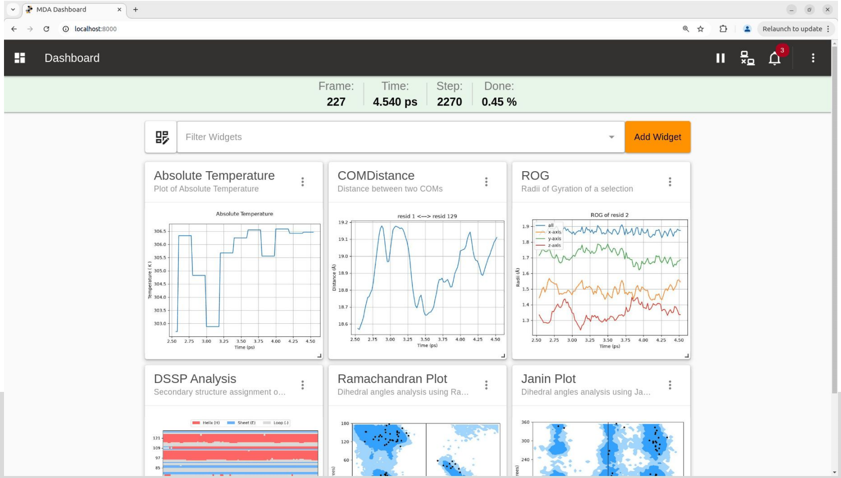
- Interactive Molecular Dynamics (IMD) - version 3 protocol
- Backward compatible with IMDv2
- Implemented in:
  - **GROMACS**, **NAMD3**, **LAMMPS**
- imdclient library
- IMDReader in MDAnalysis



mdadash - MDAnalysis Dashboard

- Python command line tool
  - Available on PyPI and conda-forge, runs in any Python environment
- Provides browser-based real-time dashboard for monitoring, tracking and analyzing running MD simulations

```
mdadash --topology topology.tpr --trajectory imd://localhost:8889
```



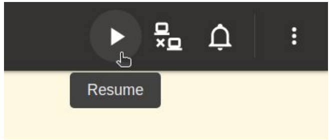
mdadash - MDAnalysis Dashboard

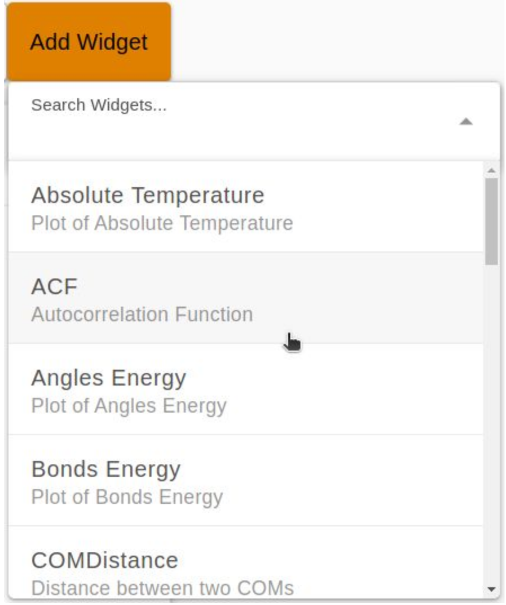
- An MDAnalysis sub-project
- Uses IMDReader introduced in MDAnalysis Release 2.10.0
  - Uses imdclient to support IMDv3 Protocol (streaming)
    - Streaming Molecular Dynamics Simulation Data for On-the-fly Processing and Analysis
- Also supports regular file based trajectories
  - Considers them as running MD simulations iterating forward
  - Uses other MDAnalysis Trajectory Readers based on format
- Started as a Google Summer of Code (GSoC) 2026 project

Google
Summer of Code

mdadash - Features

- Control simulation (Pause / Resume / Connect / Disconnect)
- Built-in Analysis Widgets
- Buffered Access and Batching
- Parallelization
- Alerts
- Persistence
  - Single JSON based state file
- Customization
  - Custom code widget
  - Custom Analysis Widgets
  - Integrated Notebook interface
- Scalable Architecture





mdadash - Features

Built-in Analysis Widgets

- Autocorrelation Function (ACF)
  - Velocity, Position, Force
- MSD
- RMSD
- DSSP
- Simulation Energies
- Native Contacts
- Dihedral angles analysis plots
- Custom user-defined code
- … and more
- See full list in documentation

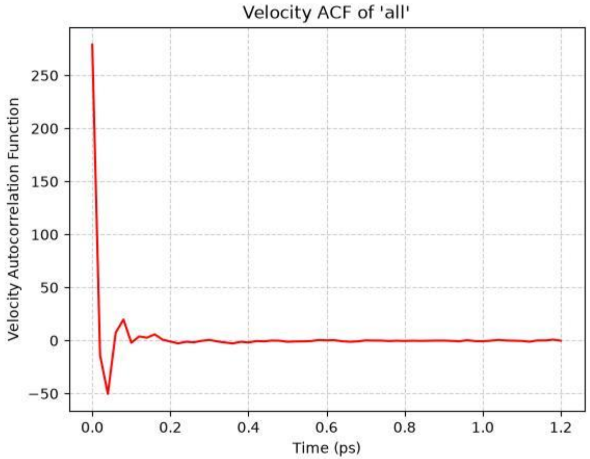
Widget	Description	Batching	Parallel
`acf`	Autocorrelation Function (ACF)	—	✅
`com_distance`	Distance between two center-of-masses	✅	✅
`contacts`	Contacts within a cutoff	✅	✅
`custom_code`	Custom user-defined code	✅	—
`dssp`	DSSP Analysis	✅	✅
`energies`	Widgets for various simulation energies	✅	—
`helix_analysis`	Helix Analysis	✅	✅
`hydrogen_bonds`	Number of Hydrogen bonds	✅	✅
`janin`	Janin plot (Dihedral angles analysis)	—	—
`msd`	MSD Analysis	—	✅
`native_contacts`	Native Contacts Analysis	✅	✅
`ramachandran`	Ramachandran plot (Dihedral angles analysis)	—	—
`rmsd`	RMSD Analysis	✅	✅
`rog`	Radii of Gyration	✅	✅

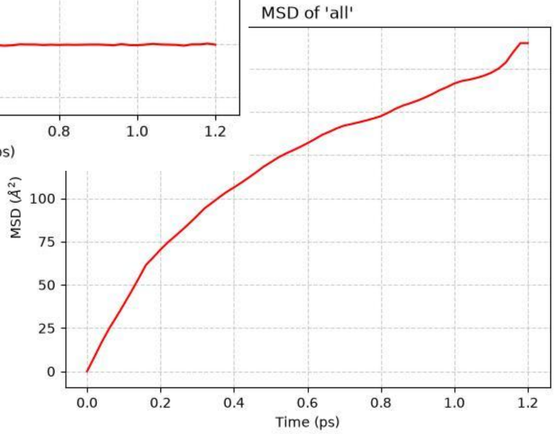
✅ Supported, ❌ Not supported, — Not applicable

mdadash - Features

Buffered Access and Batching

- Buffered access to last N timesteps
- Sliding window
- Allows for lag-time analyses
- Allows batch run of analyses
- Supports AnalysisBase-based classes
    - Not possible with regular IMDReader
    - Trajectory length equals buffer size
- See Batching in documentation

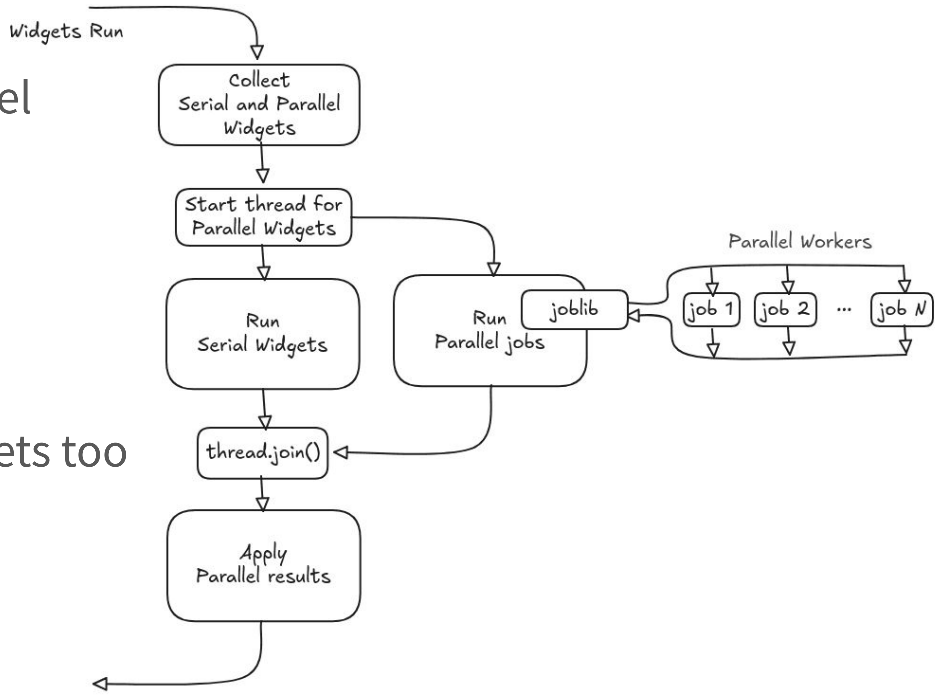




mdadash - Features

Parallelization

- Most built-in widgets can run in parallel
- Process based parallelization
  - Analyses CPU-bound
  - Uses Joblib
  - Setting to control max jobs / processes
- Can be combined with Batching
- Is supported in Custom Analysis Widgets too
- See Parallelization in documentation



mdadash - Features

Custom Code Widget

- Custom user-defined code
  - Notebook cell interface
- Setup code
  - Run once during widget creation
- Execute code
  - Run as per the widget run frequency
- A quick way to run simple custom code
- See Custom Code in documentation

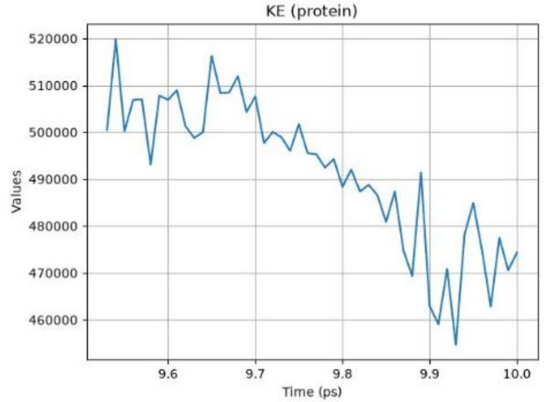
Execute code

```
ke1 = ke(u.select_atoms("protein"))
print(f"Protein KE = {ke1:.2f}")
plt1.show("KE (protein)", ke1)
```

This code will run as per the run frequency

Output

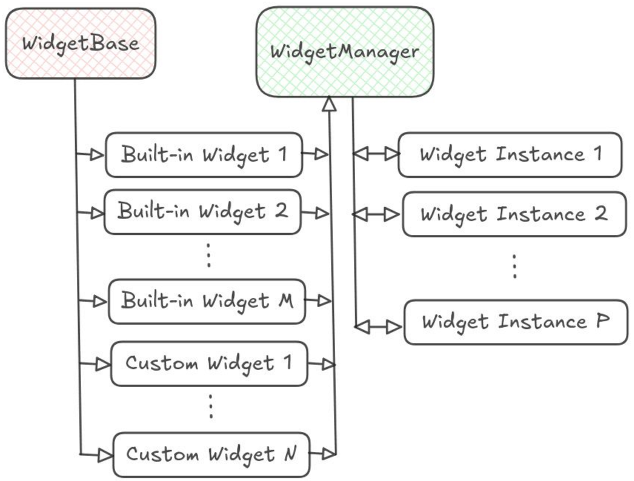
```
Protein KE = 474323.49
```



mdadash - Features

Custom Analysis Widgets

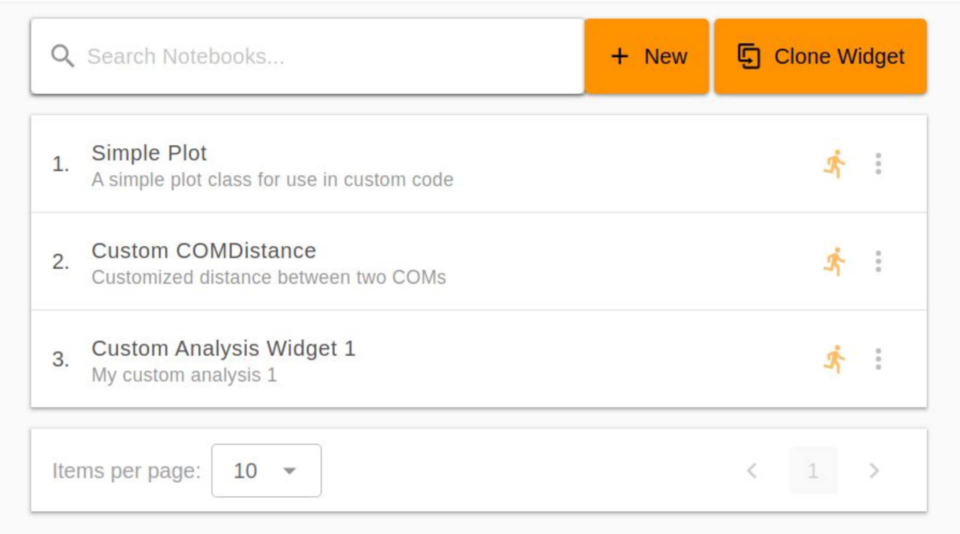
- Single common framework for Widgets
  - Same used by Built-in Analysis Widgets
  - All derive from `WidgetBase` base class
- Support for different run frequencies
  - Every-frame or Batch
- Support for different run modes
  - Serial or Parallel
- Dynamic inputs and lifecycle hooks
- See [Adding Custom Widgets]() in documentation



mdadash - Features

Integrated Notebook Interface

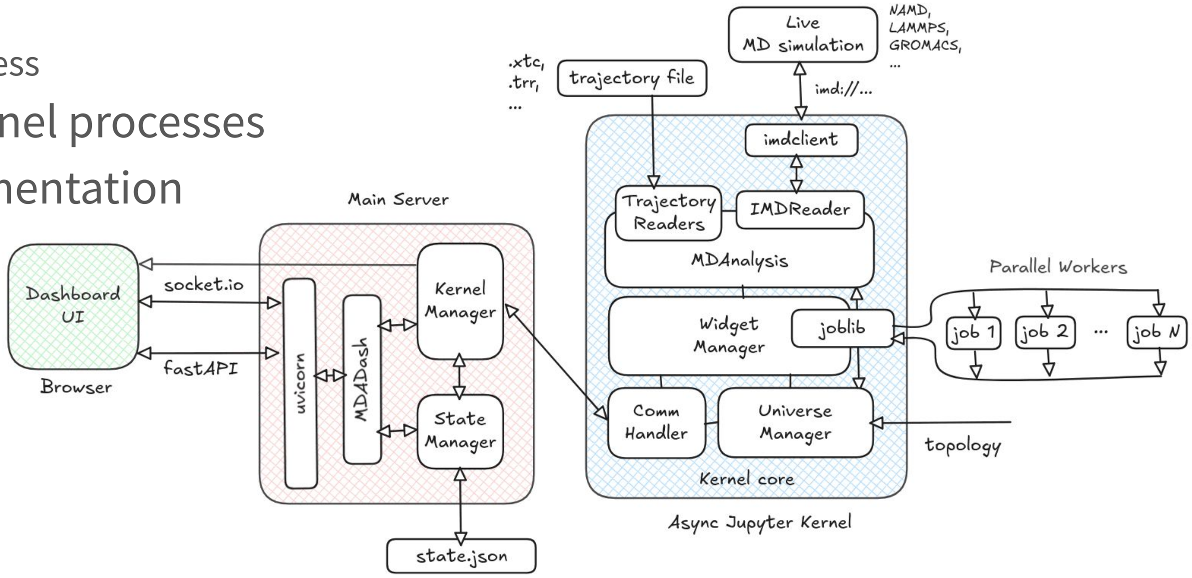
- Custom functions and Classes
  - Utils, custom plots, etc
- Notebook cells
  - Code complete support
  - Execute, re-order, delete, etc
- Run on Launch support
  - Setup custom functions / classes
  - Register custom widgets
- Clone Built-in Analysis Widget code
- Automatic refresh of widget instances



mdadash - Features

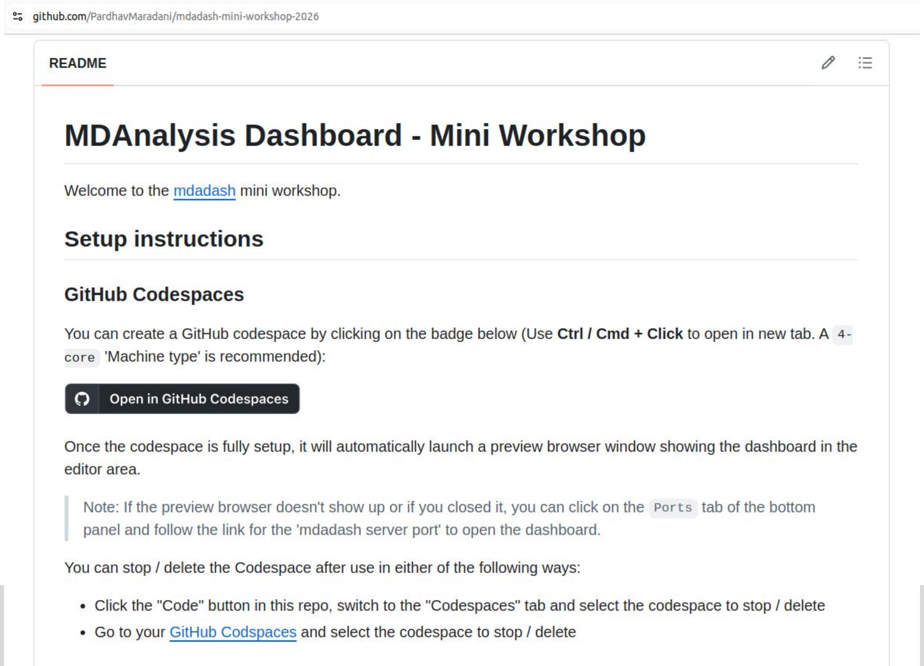
Scalable Architecture

- Two processes
  - Main Server process
  - Async Jupyter Kernel process
- Extensible to multiple kernel processes
- See Architecture in documentation



Demo and Activities

- Goto repo: https://github.com/PardhavMaradani/mdadash-mini-workshop-2026
- Follow instructions to “Open in GitHub Codespaces”
- Follow “activities.md”



github.com/PardhavMaradani/mdadash-mini-workshop-2026

README

MDAnalysis Dashboard - Mini Workshop

Welcome to the mdadash mini workshop.

Setup instructions

GitHub Codespaces

You can create a GitHub codespace by clicking on the badge below (Use **Ctrl / Cmd + Click** to open in new tab. A `4-core` 'Machine type' is recommended):

Open in GitHub Codespaces

Once the codespace is fully setup, it will automatically launch a preview browser window showing the dashboard in the editor area.

> Note: If the preview browser doesn't show up or if you closed it, you can click on the `Ports` tab of the bottom panel and follow the link for the 'mdadash server port' to open the dashboard.

You can stop / delete the Codespace after use in either of the following ways:

- Click the "Code" button in this repo, switch to the "Codespaces" tab and select the codespace to stop / delete
- Go to your GitHub Codspaces and select the codespace to stop / delete

Future Work

- Real-time Visualization
  - Mol* support in browser
  - MolecularNodes support on desktop
- Extend Notebook functionality
  - Support for Markdown, Import / Export, etc
- More Built-in Analysis Widgets
- Ensemble of simulations
- Feedback from the community

Q & A

- GitHub Repo: https://github.com/MDAnalysis/mdadash
- Documentation: https://mdadash.readthedocs.io/

MDANALYSIS

Google
Summer of Code